Slip – 1

Q1. Write a PHP script to keep track of number of times the webpage has been accessed (Use

Session Tracking).

Ans:

```
<?php

session_start();

if (isset($_SESSION['pcount'])) {

$_SESSION['pcount'] += 1;

} else {

$_SESSION['pcount'] = 1;

}

echo "You have visited this page " . $_SESSION['pcount'] . " Time(s).";

?>
```

Q2. Create 'Position_Salaries' Dataset. Build a linear regression model by identifying

independent and target variable. Split the variables into training and testing sets in a 7:3 ratio and print them. Build a simple linear regression model.

Ans:

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

# Create the Position_Salaries dataset

data = {

'Position': ['CEO', 'Chairman', 'Director', 'Senior Manager', 'Junior Manager', 'Intern'],

'Level': [1, 2, 3, 4, 5, 6],

'Salary': [50000, 80000, 110000, 150000, 200000, 250000]

}

df = pd.DataFrame(data)

# Identify independent and target variables

X = df.iloc[:, 1:2].values

y = df.iloc[:, 2].values

# Split into training and testing sets (7:3 ratio)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

print("X_train:\n", X_train)

print("y_train:\n", y_train)

print("X_test:\n", X_test)

print("y_test:\n", y_test)

# Build simple linear regression model

regressor = LinearRegression()

regressor.fit(X_train, y_train)

print("Coefficients:", regressor.coef_)

print("Intercept:", regressor.intercept_)
```

Slip – 2

Q1. Write a PHP script to change preferences (font style, font size, font color, background color) using Cookie. Display selected settings on next page and implement on third page.

Ans:

```
<!-- firstpage.html -->

<!DOCTYPE html>

<html>

<head><title>Change Preferences</title></head>

<body>

<h1>Change Preferences</h1>

<form action="secondpage.php" method="post">

<label>Font Style:</label>

<select name="fontstyle">

<option value="Arial">Arial</option>

<option value="Times New Roman">Times New Roman</option>

<option value="Verdana">Verdana</option
```

```
</select><br><br>

<label>Font Size:</label>

<select name="fontsize">

<option value="12">12</option>

<option value="14">14</option>

<option value="16">16</option>

</select><br><br>

<label>Font Color:</label>

<input type="color" name="fontcolor"><br><br>

<label>Background Color:</label>

<input type="color" name="bgcolor"><br><br>

<input type="submit" name="submit" value="Save">

</form>

</body></html>

<?php // secondpage.php

if (isset($_POST['submit'])) {

setcookie('fontstyle', $_POST['fontstyle'], time()+86400);

setcookie('fontsize', $_POST['fontsize'], time()+86400);

setcookie('fontcolor', $_POST['fontcolor'], time()+86400);

setcookie('bgcolor', $_POST['bgcolor'], time()+86400);

header('Location: thirdpage.php');

exit();

}

?>

<?php // thirdpage.php

$fontstyle = isset($_COOKIE['fontstyle']) ? $_COOKIE['fontstyle'] : 'Arial';

$fontsize = isset($_COOKIE['fontsize']) ? $_COOKIE['fontsize'] : '12';

$fontcolor = isset($_COOKIE['fontcolor']) ? $_COOKIE['fontcolor'] : '#000000';

$bgcolor = isset($_COOKIE['bgcolor']) ? $_COOKIE['bgcolor'] : '#FFFFFF';

?>

<!DOCTYPE html>

<html>

<head>

<style>

body {

font-family: <?php echo $fontstyle; ?>;

font-size: <?php echo $fontsize; ?>px;

color: <?php echo $fontcolor; ?>;

background-color: <?php echo $bgcolor; ?>;

}

</style>

</head>

<body>

<h1>Page with new settings</h1>

<p>Font style: <?php echo $fontstyle; ?>, size: <?php echo $fontsize; ?>px, color: <?php echo $fontcolor; ?>, background: <?php echo $bgcolor; ?>.</p>

</body></html>
```

Q2. Create 'Salary' Dataset. Build a linear regression model, split into training and testing sets,

build a simple linear regression model for predicting purchases.

Ans:

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

data = {

'Years_Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

'Salary': [50000, 60000, 70000, 80000, 90000,

100000, 110000, 120000, 130000, 140000]

}
```

```
df = pd.DataFrame(data)

X = df.iloc[:, 0:1].values

y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

print("X_train:\n", X_train)

print("y_train:\n", y_train)

print("X_test:\n", X_test)

print("y_test:\n", y_test)

regressor = LinearRegression()

regressor.fit(X_train, y_train)

print("Coefficients:", regressor.coef_)

print("Intercept:", regressor.intercept_)
```

Slip – 3

Q1. Write a PHP script to accept username and password. If in the first three chances username

and password entered is correct then display second form with 'Welcome message' otherwise display error message. [Use Session].

Ans:

```
<?php

session_start();

if (isset($_POST['submit'])) {

$username = $_POST['username'];

$password = $_POST['password'];

$correct_username = 'myusername';

$correct_password = 'mypassword';

if ($username == $correct_username && $password == $correct_password) {

$_SESSION['loggedin'] = true;

header('Location: welcome.php');

exit;

} else {

if (isset($_SESSION['attempts'])) {

$_SESSION['attempts']--;

} else {

$_SESSION['attempts'] = 3;

}

if ($_SESSION['attempts'] <= 0) {

echo "Maximum login attempts exceeded. Please try again later.";

} else {

echo "Invalid username or password. You have " . $_SESSION['attempts'] . " Attempt(s) left.";

}

}

}

?>

<form method="post">

<label>Username:</label>

<input type="text" name="username" required><br><br>

<label>Password:</label>

<input type="password" name="password" required><br><br>

<input type="submit" name="submit" value="Log In">

</form>
```

Q2. Create 'User' Dataset having 5 columns: UserID, Gender, Age, EstimatedSalary, Purchased.

Build a logistic regression model to predict whether a person will buy a car or not.

Ans:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

data = {

'User_ID': 'Age': 'Purchased': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

'Gender': ['Male','Male','Female','Female','Male','Male','Female','Female','Male','Female'],
```

```
[19, 35, 26, 27, 19, 27, 32, 25, 33, 45],

'Estimated_Salary': [19000, 20000, 43000, 57000, 76000, 58000, 82000, 32000, 69000, 65000],

[0, 0, 0, 1, 1, 0, 1, 0, 1, 1]

}

df = pd.DataFrame(data)

X = df.iloc[:, 2:4].values

y = df.iloc[:, 4].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

lr = LogisticRegression(random_state=0)

lr.fit(X_train, y_train)

# Predict single observation

print(lr.predict([[30, 87000]]))

# Predict multiple observations

print(lr.predict([[30, 87000], [50, 45000], [22, 30000]]))
```

Slip – 4

Q1. Write a PHP script to accept Employee details (Eno, Ename, Address) on first page,

earnings (Basic, DA, HRA) on second page, and print Employee information with Total on third

page. [Use Session].

Ans:

```
<?php // firstpage.php

session_start();

if (isset($_POST['eno'])) {

$_SESSION['eno'] = $_POST['eno'];

$_SESSION['ename'] = $_POST['ename'];

$_SESSION['address'] = $_POST['address'];

}

?>

<!DOCTYPE html>

<html><body>

<h1>Employee Details</h1>

<form method="POST" action="secondpage.php">

<label>Employee No:</label> <input type="text" name="eno"><br><br>

<label>Employee Name:</label> <input type="text" name="ename"><br><br>

<label>Address:</label> <textarea name="address"></textarea><br><br>

<input type="submit" value="Next">

</form>

</body></html>

<?php // secondpage.php

session_start();

if (isset($_POST['basic'])) {

$_SESSION['basic'] = $_POST['basic'];

$_SESSION['da'] = $_POST['da'];

$_SESSION['hra'] = $_POST['hra'];

}

?>

<!DOCTYPE html>

<html><body>

<h1>Earnings</h1>

<form method="POST" action="thirdpage.php">

<label>Basic:</label> <input type="text" name="basic"><br><br>

<label>DA:</label> <input type="text" name="da"><br><br>

<label>HRA:</label> <input type="text" name="hra"><br><br>

<input type="submit" value="Next">

</form>

</body></html>

<?php // thirdpage.php

session_start();

$total = $_SESSION['basic'] + $_SESSION['da'] + $_SESSION['hra'];

?>
```

```
<!DOCTYPE html>

<html><body>

<h1>Employee Information</h1>

<p><strong>Employee No:</strong> <?php echo $_SESSION['eno']; ?></p>

<p><strong>Employee Name:</strong> <?php echo $_SESSION['ename']; ?></p>

<p><strong>Address:</strong> <?php echo $_SESSION['address']; ?></p>

<p><strong>Basic:</strong> <?php echo $_SESSION['basic']; ?></p>

<p><strong>DA:</strong> <?php echo $_SESSION['da']; ?></p>

<p><strong>HRA:</strong> <?php echo $_SESSION['hra']; ?></p>

<p><strong>Total Earnings:</strong><?php echo $total; ?></p>

</body></html>
```

Q2. Build a simple linear regression model for Fish Species Weight Prediction.

Ans:

```
import pandas as pd

import random

from sklearn.linear_model import LinearRegression

fish_species = ['Tuna', 'Salmon', 'Trout', 'Bass', 'Sardine', 'Cod', 'Mackerel']

weights = []

for i in range(50):

fish_weight = [random.randint(1, 20) for j in range(7)]

weights.append(fish_weight)

df = pd.DataFrame(weights, columns=fish_species)

X = df.iloc[:, :-1] y = df.iloc[:, -1] # independent variables

# target variable

model = LinearRegression()

model.fit(X, y)

new_fish = [[10, 12, 15, 7, 4, 8]]

predicted_weight = model.predict(new_fish)

print("Predicted weight:", predicted_weight)
```

Slip – 5

Q1. Create XML file named "Item.xml" with item-name, item-rate, item quantity. Store details of

5 items of different types.

Ans:

```
<?xml version="1.0"?>

<!-- Item.xml -->

<items>

<item type="Electronics">

<name>Television</name>

<rate>500</rate>

<quantity>10</quantity>

</item>

<item type="Clothing">

<name>Shirt</name>

<rate>50</rate>

<quantity>20</quantity>

</item>

<item type="Grocery">

<name>Rice</name>

<rate>40</rate>

<quantity>30</quantity>

</item>

<item type="Books">

<name>Harry Potter and the Philosopher's Stone</name>

<rate>20</rate>

<quantity>50</quantity>

</item>

<item type="Sports">

<name>Football</name>

<rate>100</rate>

<quantity>5</quantity>

</item>

</items>
```

Q2. Use the Iris dataset. View basic statistical details (percentile, mean, std etc.) of Iris species.

Apply logistic regression to identify species using 4 features. Find the accuracy of the model.

Ans:

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['target'] = iris.target

print("Statistical details of Iris-setosa:")

print(df[df['target'] == 0].describe())

print("\nStatistical details of Iris-versicolor:")

print(df[df['target'] == 1].describe())

print("\nStatistical details of Iris-virginica:")

print(df[df['target'] == 2].describe())

X = df.iloc[:, :-1]

y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

logreg = LogisticRegression()

logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy of the logistic regression model:", accuracy)
```

Slip – 6

Q1. Write PHP script to read "book.xml" file into SimpleXML object. Display attributes and

elements. (simplexml_load_file() function).

Ans:

```
<?php

// book.xml

// <?xml version="1.0"?>

// <book isbn="978-3-16-148410-0" publisher="Example Publisher">

// <title>Example Book</title>

// <author>John Doe</author>

// <description>This is an example book.</description>

// </book>

$xml = simplexml_load_file("book.xml");

echo "Book attributes:<br>";

echo "ISBN: " . $xml['isbn'] . "<br>";

echo "Publisher: " . $xml['publisher'] . "<br><br>";

echo "Book elements:<br>";

echo "Title: " . $xml->title . "<br>";

echo "Author: " . $xml->author . "<br>";

echo "Description: " . $xml->description . "<br>";

?>
```

Q2. Create the following dataset in Python and convert categorical values to numeric format.

Apply the Apriori algorithm to generate frequent itemsets and association rules. Repeat with

different min_sup values. TID = {1:[bread,milk], 2:[bread,diaper,beer,eggs],

3:[milk,diaper,beer,coke], 4:[bread,milk,diaper,beer], 5:[bread,milk,diaper,coke]}

Ans:

```
import pandas as pd
```

```
from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

TID = {

1: ["bread", "milk"],

2: ["bread", "diaper", "beer", "eggs"],

3: ["milk", "diaper", "beer", "coke"],

4: ["bread", "milk", "diaper", "beer"],

5: ["bread", "milk", "diaper", "coke"]

}

transactions = [v for v in TID.values()]

te = TransactionEncoder()

te_ary = te.fit_transform(transactions)

df = pd.DataFrame(te_ary, columns=te.columns_)

for min_sup in [0.2, 0.4, 0.6]:

frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

print(f"\nMin_sup: {min_sup}")

print("Frequent Itemsets:\n", frequent_itemsets)

print("Association Rules:\n", rules)
```

Slip – 7

Q1. Write a PHP script to read "Movie.xml" file and print all Movie Titles and Actor Names using

DOMDocument Parser. Movie.xml should contain at least 5 records with Movie No, Movie Title,

Actor Name, Release Year.

Ans:

```
<?php

// PHP file

$xml = new DOMDocument();

$xml->load('Movie.xml');

$movies = $xml->getElementsByTagName('MovieInfo');

foreach ($movies as $movie) {

echo "Movie Title: " . $movie->getElementsByTagName('MovieTitle')[0]->textContent . ", ";

echo "Actor Name: " . $movie->getElementsByTagName('ActorName')[0]->textContent . "<br>";

}

?>

<?xml version="1.0"?>

<!-- Movie.xml -->

<MovieList>

<MovieInfo>

<MovieNo>1</MovieNo>

<MovieTitle>The Shawshank Redemption</MovieTitle>

<ActorName>Tim Robbins</ActorName>

<ReleaseYear>1994</ReleaseYear>

</MovieInfo>

<MovieInfo>

<MovieNo>2</MovieNo>

<MovieTitle>The Godfather</MovieTitle>

<ActorName>Marlon Brando</ActorName>

<ReleaseYear>1972</ReleaseYear>

</MovieInfo>

<MovieInfo>

<MovieNo>3</MovieNo>

<MovieTitle>The Dark Knight</MovieTitle>

<ActorName>Christian Bale</ActorName>

<ReleaseYear>2008</ReleaseYear>

</MovieInfo>

<MovieInfo>
```

<MovieNo>4</MovieNo>

<MovieTitle>The Godfather: Part II</MovieTitle>

<ActorName>Al Pacino</ActorName>

<ReleaseYear>1974</ReleaseYear>

</MovieInfo>

<MovieInfo>

<MovieNo>5</MovieNo>

<MovieTitle>12 Angry Men</MovieTitle>

<ActorName>Henry Fonda</ActorName>

<ReleaseYear>1957</ReleaseYear>

</MovieInfo>

</MovieList>

Q2. Download the Market Basket dataset. Read the dataset, display info, preprocess (drop null

values etc.), convert categorical values to numeric format. Apply Apriori algorithm to generate

frequent itemsets and association rules.

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)

df.dropna(inplace=True)

te = TransactionEncoder()

te_ary = te.fit(df.values).transform(df.values)

df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.01, use_colnames=True)

rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)

print("Frequent Itemsets:\n", frequent_itemsets)

print("\nAssociation Rules:\n", rules)
```

Slip – 8

Q1. Write a JavaScript to display message 'Exams are near, have you started preparing for?'

(use alert Box) and accept any two numbers from user and display addition. (Use Prompt and

Confirm box).

Ans:

```
// Display message using alert box

alert('Exams are near, have you started preparing for?');

// Accept two numbers from user

let num1 = prompt('Enter first number:');

let num2 = prompt('Enter second number:');

let confirmMsg = `Are you sure you want to add ${num1} and ${num2}?`;

let confirmResult = confirm(confirmMsg);

if (confirmResult) {

num1 = parseInt(num1);

num2 = parseInt(num2);

let sum = num1 + num2;

alert(`The sum of ${num1} and ${num2} is ${sum}.`);

}
```

Q2. Download the groceries dataset. Read the dataset, display info, preprocess (drop null

values etc.), convert categorical values to numeric format. Apply Apriori algorithm to generate

frequent itemsets and association rules.

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules
```

```
df = pd.read_csv('market_basket.csv')

df.dropna(inplace=True)

te = TransactionEncoder()

te_ary = te.fit(df.values).transform(df.values)

df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.01, use_colnames=True)

rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)

print("Dataset information:")

print(df.info())

print("\nFrequent itemsets:\n", frequent_itemsets)

print("\nAssociation rules:\n", rules)
```

Slip – 9

Q1. Write a JavaScript function to validate username and password for a membership form.

Ans:

```
function validateForm() {

var username = document.forms["membershipForm"]["username"].value;

var password = document.forms["membershipForm"]["password"].value;

if (username == "") {

alert("Username must be filled out");

return false;

}

if (password == "") {

alert("Password must be filled out");

return false;

}

return true;

}
```

Q2. Create your own transactions dataset and apply the Apriori process on your dataset.

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

transactions = [

['item1', 'item2', 'item3'],

['item2', 'item3'],

['item1', 'item2', 'item4'],

['item1', 'item4'],

['item2', 'item3', 'item4'],

['item1', 'item3', 'item4'],

['item1', 'item2'],

['item1', 'item3'],

['item3', 'item4'],

['item2', 'item4']

]

te = TransactionEncoder()

te_ary = te.fit_transform(transactions)

df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)

assoc_rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

print(frequent_itemsets)

print(assoc_rules)
```

Slip – 10

Q1. Create a HTML file to insert text before and after a Paragraph using jQuery. [Hint: Use

before() and after()].

Ans:

```
<!DOCTYPE html>

<html>

<head>
```

```
</head>

<title>Insert text before and after paragraph using jQuery</title>

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

<body>

<h1>Insert text before and after paragraph using jQuery</h1>

<p>This is a paragraph.</p>

<script>

$(document).ready(function() {

$("p").before("Text inserted before the paragraph.");

$("p").after("Text inserted after the paragraph.");

});

</script>

</body>

</html>
```

Q2. Create the following dataset in Python and convert categorical values to numeric format. Apply the Apriori algorithm. Repeat with different min_sup values. TID = {1:[eggs,milk,bread],

2:[eggs,apple], 3:[milk,bread], 4:[apple,milk], 5:[milk,apple,bread]}

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

dataset = {

1: ["eggs", "milk", "bread"],

2: ["eggs", "apple"],

3: ["milk", "bread"],

4: ["apple", "milk"],

5: ["milk", "apple", "bread"]

}

te = TransactionEncoder()

te_ary = te.fit(dataset.values()).transform(dataset.values())

df = pd.DataFrame(te_ary, columns=te.columns_)

for min_sup in [0.4, 0.6, 0.8]:

frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

assoc_rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)

print(f"\nFrequent Itemsets (min_sup={min_sup}):\n", frequent_itemsets)

print("Association Rules:\n", assoc_rules)
```

Slip – 11

Q1. Write a JavaScript program to accept name of student, change font color to red, font size to

18 if student name is present; otherwise on clicking empty textbox display image which

changes its size. (Use onblur, onload, onmouseover, onmousedown, onmouseup)

Ans:

```
<!DOCTYPE html>

<html>

<head>

<title>JavaScript Example</title>

<style>

#name { font-size: 14px; color: black; }

</style>

</head>

<body>

<input type="text" id="name"

onblur="changeStyle()"

onmouseover="changeSize()"

onmouseout="resetSize()"

onmousedown="changeColor()"

onmouseup="resetColor()">

<img id="img" src="https://via.placeholder.com/150" onload="changeImageSize()">
```

```
<script>

function changeStyle() {

let name = document.getElementById("name").value;

if (name) {

document.getElementById("name").style.fontSize = "18px";

document.getElementById("name").style.color = "red";

} else {

document.getElementById("img").style.display = "block";

}

}

function changeSize() { document.getElementById("name").style.fontSize = "16px"; }

function resetSize() { document.getElementById("name").style.fontSize = "14px"; }

function changeColor() { document.getElementById("name").style.color = "blue"; }

function resetColor() { document.getElementById("name").style.color = "red"; }

function changeImageSize() {

document.getElementById("img").style.width = "200px";

document.getElementById("img").style.height = "200px";

}

</script>

</body>

</html>
```

Q2. Create the following dataset in Python and apply Apriori with different min_sup values. TID

= {1:[butter,bread,milk], 2:[butter,flour,milk,sugar], 3:[butter,eggs,milk,salt], 4:[eggs],

5:[butter,flour,milk,salt]}

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

dataset = [

['butter', 'bread', 'milk'],

['butter', 'flour', 'milk', 'sugar'],

['butter', 'eggs', 'milk', 'salt'],

['eggs'],

['butter', 'flour', 'milk', 'salt']

]

te = TransactionEncoder()

te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)

for min_sup in [0.4, 0.3, 0.2]:

frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

print(f"\nFrequent Itemsets (min_sup={min_sup}):\n", frequent_itemsets)

print("Association Rules:\n", rules)
```

Slip – 12

Q1. Write AJAX program to read contact.dat file and print contents in tabular format when user

clicks on Print button. contact.dat should contain srno, name, residence number, mobile

number, Address. [At least 3 records]

Ans:

```
<!-- HTML file -->

<!DOCTYPE html>

<html>

<head>

<title>Contact List</title>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
```

```
<script src="script.js"></script>

</head>

<body>

<button id="printBtn">Print Contacts</button><br><br>

<table id="contactTable">

<thead>

<tr>

<th>Sr.No.</th><th>Name</th>

<th>Residence Number</th><th>Mobile Number</th><th>Address</th>

</tr>

</thead>

<tbody></tbody>

</table>

</body>

</html>

// script.js (AJAX file)

$(document).ready(function() {

$("#printBtn").click(function() {

$.ajax({

url: "contact.dat",

dataType: "text",

success: function(data) {

var lines = data.split("\n");

var tableRows = "";

for (var i = 0; i < lines.length; i++) {

var columns = lines[i].split(",");

if (columns.length == 5) {

tableRows += "<tr>";

for (var j = 0; j < columns.length; j++) {

tableRows += "<td>" + columns[j] + "</td>";

}

tableRows += "</tr>";

}

}

$("#contactTable tbody").html(tableRows);

error: function(jqXHR, textStatus, errorThrown) {

alert("Error: " + errorThrown);

},

}

});

});

});
```

Q2. Create 'heights-and-weights' Dataset. Build a linear regression model, split into training

and testing sets, build a simple linear regression model for predicting weight from height.

Ans:

```
import numpy as np

import pandas as pd

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

heights = np.random.normal(170, 10, 10)

weights = np.random.normal(70, 5, 10)

dataset = pd.DataFrame({'Height': heights, 'Weight': weights})

X_train, X_test, y_train, y_test = train_test_split(

dataset['Height'], dataset['Weight'], test_size=0.2, random_state=42

)

lr_model = LinearRegression()

lr_model.fit(X_train.values.reshape(-1, 1), y_train)

print('Model Coefficients:', lr_model.coef_)

y_pred = lr_model.predict(X_test.values.reshape(-1, 1))

print('Predictions:', y_pred)
```

Slip – 13

Q1. Write an AJAX program where the user types their name in a textbox and the server

responds while typing. If name is empty: 'Stranger, please tell me your name!'. If name is

Rohit/Virat/Dhoni/Ashwin/Harbhajan: 'Hello, master!'. Otherwise: name + ', I don't know you!'.

Ans:

```
<!-- HTML file -->

<!DOCTYPE html>

<html>

<head>

<title>AJAX Program</title>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

</head>

<body>

<label>Enter your name:</label>

<input type="text" id="name" name="name">

<div id="response"></div>

<script src="ajax.js"></script>

</body>

</html>

// ajax.js

$(document).ready(function() {

$('#name').on('input', function() {

var name = $(this).val();

$.ajax({

url: 'server.php',

type: 'POST',

data: { name: name },

success: function(response) {

$('#response').html(response);

}

});

});

});

<?php // server.php

$name = $_POST['name'];

if (empty($name)) {

echo 'Stranger, please tell me your name!';

} elseif (in_array($name, ['Rohit', 'Virat', 'Dhoni', 'Ashwin', 'Harbhajan'])) {

echo 'Hello, master!';

} else {

echo $name . ", I don't know you!";

}

?>
```

Q2. Download nursery dataset from UCI. Build a linear regression model by identifying

independent and target variable. Split into training and testing sets. Build a simple linear

regression model.

Ans:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/nursery/nursery.data"

names = ['parents', 'has_nurs', 'form', 'children', 'housing', 'finance', 'social', 'health', 'class']

dataset = pd.read_csv(url, names=names)

X = dataset.drop('class', axis=1)

y = dataset['class']

X = pd.get_dummies(X) # Convert categorical to numerical

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()

model.fit(X_train, y_train)

print("Intercept:", model.intercept_)

print("Coefficients:", model.coef_)

y_pred = model.predict(X_test)
```

```
mse = np.mean((y_test - y_pred) ** 2)

print("MSE:", mse)
```

Slip – 14

Q1. Create TEACHER table (tno, tname, qualification, salary). Write an AJAX program to select

a teacher's name and print the selected teacher's details.

Ans:

```
<!-- HTML / JS file -->

<!DOCTYPE html>

<html>

<head>

<title>Teacher Details</title>

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

</head>

<body>

<select id="teacher-list">

<option value="">-- Select Teacher --</option>

<option value="1">John Doe</option>

<option value="2">Jane Smith</option>

<option value="3">Bob Johnson</option>

</select>

<button id="submit-btn">Get Details</button>

<div id="details"></div>

<script>

$(document).ready(function() {

$('#submit-btn').click(function() {

var tno = $('#teacher-list').val();

if (tno == '') { alert('Please select a teacher.'); return; }

$.ajax({

url: 'teacherdetails.php',

method: 'POST',

data: { tno: tno },

success: function(response) { $('#details').html(response); },

error: function(xhr, status, error) { console.log(xhr.responseText); }

});

});

});

</script>

</body>

</html>

<?php // teacherdetails.php

$conn = mysqli_connect("localhost", "username", "password", "database_name");

if (!$conn) { die("Connection failed: " . mysqli_connect_error()); }

if (isset($_POST['tno'])) {

$tno = $_POST['tno'];

$sql = "SELECT * FROM TEACHER WHERE tno='$tno'";

$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) > 0) {

$row = mysqli_fetch_assoc($result);

echo "Teacher Name: " . $row['tname'] . "<br>";

echo "Qualification: " . $row['qualification'] . "<br>";

echo "Salary: " . $row['salary'] . "<br>";

} else {

echo "No data found.";

}

}

mysqli_close($conn);

?>
```

Q2. Create the following dataset in Python and apply Apriori with different min_sup values. TID

= {1:[apple,mango,banana], 2:[mango,banana,cabbage,carrots], 3:[mango,banana,carrots],

4:[mango,carrots]}

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

TID = {

1: ["apple", "mango", "banana"],

2: ["mango", "banana", "cabbage", "carrots"],

3: ["mango", "banana", "carrots"],

4: ["mango", "carrots"]

}

te = TransactionEncoder()

te_ary = te.fit([TID[i] for i in TID]).transform([TID[i] for i in TID])

df = pd.DataFrame(te_ary, columns=te.columns_)

for min_sup in [0.25, 0.5, 0.75]:

frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

print(f"\nFrequent itemsets (min_sup={min_sup}):\n", frequent_itemsets)
```

Slip – 15

Q1. Write an AJAX program to fetch suggestions while user is typing in a textbox (like Google

suggestions). Create an array of suggestions and display matching strings.

Ans:

```
<!DOCTYPE html>

<html>

<head>

<title>AJAX Auto Suggestions Example</title>

<script>

function fetchSuggestions(str) {

if (str.length == 0) {

document.getElementById("suggestions").innerHTML = "";

return;

}

var suggestions = [

"apple", "banana", "cherry", "dates", "elderberry",

"fig", "grape", "honeydew", "kiwi", "lemon"

];

var matches = [];

for (var i = 0; i < suggestions.length; i++) {

if (suggestions[i].toLowerCase().startsWith(str.toLowerCase())) {

matches.push(suggestions[i]);

}

}

if (matches.length > 0) {

document.getElementById("suggestions").innerHTML = matches.join("<br>");

} else {

document.getElementById("suggestions").innerHTML = "No suggestions found";

}

}

</script>

</head>

<body>

<input type="text" onkeyup="fetchSuggestions(this.value)">

<div id="suggestions"></div>

</body>

</html>
```

Q2. Create the following dataset in Python and apply Apriori with different min_sup values. No |

Company | Model | Year: 1-Tata Nexon 2017, 2-MG Astor 2021, 3-Kia Seltos 2019, 4-Hyundai

Creta 2015

Ans:

```
import pandas as pd
```

```
from mlxtend.frequent_patterns import apriori, association_rules

data = {

'No': [1, 2, 3, 4],

'Company': ['Tata', 'MG', 'Kia', 'Hyundai'],

'Model': ['Nexon', 'Astor', 'Seltos', 'Creta'],

'Year': [2017, 2021, 2019, 2015]

}

df = pd.DataFrame(data)

# Convert categorical values to numeric

df['Company'] = pd.Categorical(df['Company']).codes

df['Model'] = pd.Categorical(df['Model']).codes

print(df)

frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)

print(frequent_itemsets)

assoc_rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

print(assoc_rules)
```

Slip – 16

Q1. Write an AJAX program to get book details from an XML file when user selects a book

name. Create XML file storing details of books (title, author, year, price).

Ans:

```
<?xml version="1.0"?>

<!-- book_details.xml -->

<books>

<book>

<title>The Great Gatsby</title>

<author>F. Scott Fitzgerald</author>

<year>1925</year>

<price>10.99</price>

</book>

<book>

<title>To Kill a Mockingbird</title>

<author>Harper Lee</author>

<year>1960</year>

<price>8.99</price>

</book>

<book>

<title>1984</title>

<author>George Orwell</author>

<year>1949</year>

<price>6.99</price>

</book>

<book>

<title>Pride and Prejudice</title>

<author>Jane Austen</author>

<year>1813</year>

<price>7.99</price>

</book>

</books>

<!DOCTYPE html>

<html>

<head>

<title>Book Details</title>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

<script>

$(document).ready(function() {

$("select").change(function() {

var book = $(this).val();

$.ajax({

url: "book_details.xml",

dataType: "xml",

success: function(xml) {

$(xml).find('book').each(function() {

var title = $(this).find('title').text();

if (title == book) {

var author = $(this).find('author').text();

var year = $(this).find('year').text();
```

```
var price = $(this).find('price').text();

$("#details").html("Author: " + author +

"<br>Year: " + year +

"<br>Price: " + price);

}

});

}

});

});

});

</script>

</head>

<body>

<select>

<option>Select a book</option>

<option>The Great Gatsby</option>

<option>To Kill a Mockingbird</option>

<option>1984</option>

<option>Pride and Prejudice</option>

</select>

<div id="details"></div>

</body>

</html>
```

Q2. Consider any text paragraph. Preprocess the text to remove special characters and digits.

Generate the summary using extractive summarization process.

Ans:

```
import re

import nltk

from nltk.corpus import stopwords

from nltk.tokenize import sent_tokenize, word_tokenize

from heapq import nlargest

text = ("Natural language processing (NLP) is a subfield of linguistics, computer science, "

"information engineering, and artificial intelligence concerned with the interactions between "

"computers and human languages. Challenges in NLP frequently involve speech recognition, "

"natural language understanding, and natural language generation.")

# Remove special characters and digits

text = re.sub('[^a-zA-Z ]', '', text)

sentences = sent_tokenize(text)

stop_words = set(stopwords.words('english'))

words = []

for sentence in sentences:

words.extend(word_tokenize(sentence))

words = [w.lower() for w in words if w.lower() not in stop_words]

word_freq = nltk.FreqDist(words)

sentence_scores = {}

for sentence in sentences:

for word in word_tokenize(sentence.lower()):

if word in word_freq:

if len(sentence.split(' ')) < 30:

sentence_scores[sentence] = sentence_scores.get(sentence, 0) + word_freq[word]

summary_sentences = nlargest(3, sentence_scores, key=sentence_scores.get)

summary = ' '.join(summary_sentences)

print(summary)
```

Slip – 17

Q1. Write a JavaScript Program to show 'Hello Good Morning' message on load event using

alert box and display the Student registration form.

Ans:

```
<!DOCTYPE html>

<html>

<head>

<title>Student Registration Form</title>

<script>

window.onload = function() {

alert("Hello Good Morning!");

};

</script>

</head>

<body>

<form>

<h1>Student Registration Form</h1>

<label>Name:</label>

<input type="text" name="name" required><br><br>

<label>Email:</label>

<input type="email" name="email" required><br><br>

<label>Phone:</label>

<input type="tel" name="phone" required><br><br>

<label>Address:</label>

<textarea name="address" required></textarea><br><br>

<input type="submit" value="Submit">

</form>

</body>

</html>
```

Q2. Consider the text paragraph: "So, keep working. Keep striving. Never give up. Fall down

seven times, get up eight..." Preprocess the text to remove special characters and digits.

Generate the summary using extractive summarization.

Ans:

```
import re

from nltk.tokenize import sent_tokenize

text = ("So, keep working. Keep striving. Never give up. Fall down seven times, get up eight. "

"Ease is a greater threat to progress than hardship. "

"So, keep moving, keep growing, keep learning. See you at work.")

# Remove special characters and digits

text = re.sub('[^A-Za-z ]+', '', text)

sentences = sent_tokenize(text)

# Score sentences by word count

scores = {}

for sentence in sentences:

scores[sentence] = len(sentence.split())

sorted_sentences = sorted(scores.items(), key=lambda x: x[1], reverse=True)

summary_sentences = [s[0] for s in sorted_sentences[:2]]

summary = " ".join(summary_sentences)

print(summary)
```

Slip – 18

Q1. Write a JavaScript Program to print Fibonacci numbers on onclick event.

Ans:

```
<!DOCTYPE html>

<html>

<head>

<title>Fibonacci Numbers</title>

<script>

function generateFibonacci() {

var input = document.getElementById("inputNumber").value;

var output = document.getElementById("output");

var n = parseInt(input);

var fib = [];

fib[0] = 0;

fib[1] = 1;

for (var i = 2; i <= n; i++) {
```

```
fib[i] = fib[i-1] + fib[i-2];

}

output.innerHTML = "Fibonacci sequence up to " + n + ": " + fib.join(", ");

}

</script>

</head>

<body>

<h1>Fibonacci Numbers</h1>

<p>Enter a number:</p>

<input type="text" id="inputNumber">

<button onclick="generateFibonacci()">Generate Fibonacci</button>

<p id="output"></p>

</body>

</html>
```

Q2. Consider any text paragraph. Remove stopwords. Tokenize the paragraph to extract words and sentences. Calculate word frequency distribution and plot frequencies. Plot the word cloud.

Ans:

```
# Install libraries (run once)

# pip install nltk matplotlib wordcloud

import nltk

from nltk.corpus import stopwords

from nltk.tokenize import word_tokenize, sent_tokenize

from nltk.probability import FreqDist

import matplotlib.pyplot as plt

from wordcloud import WordCloud

nltk.download('stopwords')

nltk.download('punkt')

text = ("Lorem ipsum dolor sit amet, consectetur adipiscing elit. "

"Sed tristique ante et velit vestibulum, vel pharetra orci iaculis. "

"Nullam mattis risus quis augue tincidunt rhoncus.")

words = word_tokenize(text.lower())

sentences = sent_tokenize(text)

stop_words = set(stopwords.words('english'))

filtered_words = [w for w in words if w.casefold() not in stop_words]

fdist = FreqDist(filtered_words)

fdist.plot(30, cumulative=False)

plt.show()

wordcloud = WordCloud(width=800, height=800,

background_color='white',

stopwords=stop_words,

min_font_size=10).generate(text)

plt.figure(figsize=(8, 8), facecolor=None)

plt.imshow(wordcloud)

plt.axis("off")

plt.tight_layout(pad=0)

plt.show()
```

Slip – 19

Q1. Write a JavaScript Program to validate username and password on onSubmit event.

Ans:

```
<!DOCTYPE html>

<html>

<head>

<title>Validate Username and Password</title>

<script>

function validateForm() {

var username = document.forms["myForm"]["username"].value;

var password = document.forms["myForm"]["password"].value;
```

```
if (username == "") {

alert("Username must be filled out");

return false;

}

if (password == "") {

alert("Password must be filled out");

return false;

}

}

</script>

</head>

<body>

<h2>Validate Username and Password</h2>

<form name="myForm" onsubmit="return validateForm()" method="post">

<label>Username:</label>

<input type="text" name="username"><br><br>

<label>Password:</label>

<input type="password" name="password"><br><br>

<input type="submit" value="Submit">

</form>

</body>

</html>
```

Q2. Download movie_review.csv from Kaggle to perform sentiment analysis and create a word

cloud.

Ans:

```
import pandas as pd

from textblob import TextBlob

from wordcloud import WordCloud, STOPWORDS

import matplotlib.pyplot as plt

df = pd.read_csv('movie_review.csv')

# Sentiment analysis using TextBlob

df['Sentiment'] = df['Review'].apply(lambda x: TextBlob(x).sentiment.polarity)

# Positive reviews only

pos_df = df[df['Sentiment'] > 0.2]

wordcloud = WordCloud(

width=800, height=800,

background_color='white',

stopwords=STOPWORDS,

min_font_size=10

).generate(' '.join(pos_df['Review']))

plt.figure(figsize=(8, 8), facecolor=None)

plt.imshow(wordcloud)

plt.axis("off")

plt.tight_layout(pad=0)

plt.show()
```

Slip – 20

Q1. Create a student.xml file containing at least 5 student records.

Ans:

```
<?xml version="1.0"?>

<students>

<student>

<name>John Doe</name> <age>21</age> <gender>Male</gender>

<major>Computer Science</major> <gpa>3.8</gpa>

</student>

<student>

<name>Jane Smith</name> <age>19</age> <gender>Female</gender>

<major>Business</major> <gpa>3.5</gpa>

</student>

<student>

<name>Tom Johnson</name> <age>20</age> <gender>Male</gender>

<major>Engineering</major> <gpa>3.2</gpa>
```

```
</student>

<student>

<name>Sara Lee</name>
<age>22</age>
<gender>Female</gender>

<major>Psychology</major>
<gpa>3.6</gpa>

</student>

<student>

<name>Mike Brown</name>
<age>18</age>
<gender>Male</gender>

<major>Education</major>
<gpa>3.4</gpa>

</student>

</students>
```

Q2. Consider the text paragraph: "Hello all, Welcome to Python Programming Academy..."

Remove the stopwords.

Ans:

```
import nltk

from nltk.corpus import stopwords

nltk.download('stopwords')

nltk.download('punkt')

text = ("Hello all, Welcome to Python Programming Academy. "

"Python Programming Academy is a nice platform to learn new programming skills. "

"It is difficult to get enrolled in this Academy.")

tokens = nltk.word_tokenize(text)

stop_words = set(stopwords.words('english'))

filtered_tokens = [word for word in tokens if not word.lower() in stop_words]

print(filtered_tokens)
```

Slip – 21

Q1. Add a JavaScript file in CodeIgniter. The JavaScript code should check whether a number

is Positive or Negative.

Ans:

```
<!-- HTML file (view) -->

<!DOCTYPE html>

<html>

<head>

<title>Number Check</title>

<script src="<?php echo base_url('js/checkNumber.js'); ?>"></script>

</head>

<body>

<h1>Number Check</h1>

<p>Enter a number to check:</p>

<input type="number" id="num"/>

<button onclick="checkNumber(document.getElementById('num').value)">Check</button>

</body>

</html>

// js/checkNumber.js

function checkNumber(num) {

if (num > 0) {

alert("The number is positive.");

} else if (num < 0) {

alert("The number is negative.");

} else {

alert("The number is zero.");

}

}
```

Q2. Build a simple linear regression model for User Data.

Ans:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

import matplotlib.pyplot as plt

data = pd.read_csv('user_data.csv')

data.dropna(inplace=True)
```

```
X = data['age'].values.reshape(-1, 1)

y = data['income'].values.reshape(-1, 1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

regressor = LinearRegression()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)

print("R-squared:", r2)

plt.scatter(X_test, y_test, color='gray')

plt.plot(X_test, y_pred, color='red', linewidth=2)

plt.show()
```

Slip – 22

Q1. Create a table student having attributes (rollno, name, class). Using CodeIgniter, connect to

the Database and insert 5 records in it.

Ans:

```
<?php

$conn = pg_connect("host=localhost dbname=your_database user=your_username password=your_password");

if (!$conn) { echo "Connection failed."; exit; }

// Create student table

$query = "CREATE TABLE student (

rollno INTEGER PRIMARY KEY,

name VARCHAR(50) NOT NULL,

class VARCHAR(10) NOT NULL

)";

$result = pg_query($conn, $query);

if (!$result) {

echo "Error creating table: " . pg_last_error($conn); exit;

} else {

echo "Table created successfully.<br>";

}

// Insert 5 records

$insert_query = "INSERT INTO student (rollno, name, class) VALUES

(1, 'John Doe', '10A'),

(2, 'Jane Smith', '9B'),

(3, 'Bob Johnson','11C'),

(4, 'Sarah Lee', '12D'),

(5, 'Tom Brown', '8E')";

$insert_result = pg_query($conn, $insert_query);

if (!$insert_result) {

echo "Error inserting records: " . pg_last_error($conn);

} else {

echo "Records inserted successfully.";

}

pg_close($conn);

?>
```

Q2. Consider any text paragraph. Remove the stopwords.

Ans:

```
import nltk

from nltk.corpus import stopwords

from nltk.tokenize import word_tokenize

text = ("Hello all, Welcome to Python Programming Academy. "

"Python Programming Academy is a nice platform to learn new programming skills. "

"It is difficult to get enrolled in this Academy.")

words = word_tokenize(text)

stop_words = set(stopwords.words('english'))
```

```
filtered_words = [w for w in words if w.casefold() not in stop_words]

filtered_sentence = ' '.join(filtered_words)

print(filtered_sentence)
```

Slip – 23

Q1. Create a table student having attributes (rollno, name, class) containing at least 5 records.

Using CodeIgniter, display all its records.

Ans:

```
<?php

$conn = pg_connect("host=localhost dbname=your_database user=your_username password=your_password");

if (!$conn) { echo "Connection failed."; exit; }

// (Insert records as in Slip 22...)

function display_records($table_name) {

$conn = pg_connect("host=localhost dbname=your_database user=your_username password=your_password");

if (!$conn) { echo "Connection failed."; exit; }

$query = "SELECT * FROM " . $table_name;

$result = pg_query($conn, $query);

if (!$result) { echo "Error: " . pg_last_error($conn); exit; }

echo "<table>";

echo "<tr><th>Roll No</th><th>Name</th><th>Class</th></tr>";

while ($row = pg_fetch_assoc($result)) {

echo "<tr><td>" . $row['rollno'] . "</td>"

. "<td>" . $row['name'] . "</td>"

. "<td>" . $row['class'] . "</td></tr>";

}

echo "</table>";

pg_close($conn);

}

display_records('student');

?>
```

Q2. Consider any text paragraph. Preprocess the text to remove any special characters and

digits.

Ans:

```
import re

text = "Hello, #world 123! This is a sample text paragraph. It contains special characters and 5 digits."

# Remove special characters and digits

processed_text = re.sub(r'[^a-zA-Z\s]', '', text)

print(processed_text)
```

Slip – 24

Q1. Write a PHP script to create student.xml file which contains student rollno, name, address, college and course. Print student details of a specific course in tabular format after accepting

course as input.

Ans:

```
<?php

$students = [

['rollno'=>1,'name'=>'John Doe', 'address'=>'123 Main St','college'=>'ABC College','course'=>'Compute

['rollno'=>2,'name'=>'Jane Smith', 'address'=>'456 Main St','college'=>'DEF College','course'=>'Informa

['rollno'=>3,'name'=>'Bob Johnson','address'=>'789 Main St','college'=>'GHI College','course'=>'Busines

['rollno'=>4,'name'=>'Sarah Lee', 'address'=>'101 Main St','college'=>'JKL College','course'=>'Marketi

['rollno'=>5,'name'=>'Tom Brown', 'address'=>'121 Main
```

```
St','college'=>'MNO College','course'=>'Compute

];

$xml = new SimpleXMLElement('<students></students>');

foreach ($students as $student) {

$el = $xml->addChild('student');

$el->addChild('rollno', $student['rollno']);

$el->addChild('name', $student['name']);

$el->addChild('address', $student['address']);

$el->addChild('college', $student['college']);

$el->addChild('course', $student['course']);

}

$xml->asXML('student.xml');

$course = isset($_POST['course']) ? $_POST['course'] : '';

$xml = simplexml_load_file('student.xml');

$filtered = $xml->xpath("//student[course='$course']");

echo "<table border='1'>";

echo "<tr><th>Roll No</th><th>Name</th><th>Address</th><th>College</th><th>Course</th></tr>";

foreach ($filtered as $s) {

echo "<tr><td>{$s->rollno}</td><td>{$s->name}</td><td>{$s->address}</td>

<td>{$s->college}</td><td>{$s->course}</td></tr>";

}

echo "</table>";

?>
```

Q2. Read the YouTube India videos dataset. Perform data cleaning. Find total views, total likes, total dislikes and comment count.

Ans:

```
import pandas as pd

df = pd.read_csv('INvideos.csv')

# Drop columns not required

df = df.drop(['video_id','trending_date', 'channel_title','category_id',

'publish_time','tags','thumbnail_link',

'comments_disabled','ratings_disabled','video_error_or_removed'], axis=1)

# Convert to integer

df[['views','likes','dislikes','comment_count']] = (

df[['views','likes','dislikes','comment_count']].astype(int)

)

print('Total Views:', df['views'].sum())

print('Total Likes:', df['likes'].sum())

print('Total Dislikes:', df['dislikes'].sum())

print('Total Comments:', df['comment_count'].sum())
```

Slip – 25

Q1. Write a script to create "cricket.xml" file with multiple elements. Add multiple elements for

country="India" with category field.

Ans:

```
<?php

$doc = new DOMDocument();

$cricketTeam = $doc->createElement("CricketTeam");

// Australia

$teamAustralia = $doc->createElement("Team");

$teamAustralia->setAttribute("country", "Australia");

$teamAustralia->appendChild($doc-
```

```
>createElement("player", "Steve Smith"));

$teamAustralia->appendChild($doc->createElement("runs", "7090"));

$teamAustralia->appendChild($doc->createElement("wicket", "17"));

$cricketTeam->appendChild($teamAustralia);

// India

$teamIndia = $doc->createElement("Team");

$teamIndia->setAttribute("country", "India");

$teamIndia->appendChild($doc->createElement("player", "Virat Kohli"));

$teamIndia->appendChild($doc->createElement("runs", "12169"));

$teamIndia->appendChild($doc->createElement("wicket", "4"));

$teamIndia->appendChild($doc->createElement("category", "Captain"));

$cricketTeam->appendChild($teamIndia);

$doc->appendChild($cricketTeam);

$doc->save("cricket.xml");

echo "Elements added successfully!";

?>
```

Q2. Read YouTube COVID-19 comments dataset. Perform data cleaning. Tokenize comments in

words. Perform sentiment analysis and find percentage of positive, negative and neutral

comments.

Ans:

```
import pandas as pd

import nltk

from nltk.sentiment.vader import SentimentIntensityAnalyzer

df = pd.read_csv('covid_2021_1.csv')

df.dropna(inplace=True)

df.drop_duplicates(subset='Comment', inplace=True)

nltk.download('punkt')

df['tokens'] = df['Comment'].apply(nltk.word_tokenize)

nltk.download('vader_lexicon')

sia = SentimentIntensityAnalyzer()

df['sentiment'] = df['Comment'].apply(lambda x: sia.polarity_scores(x)['compound'])

total = len(df)

positive = len(df[df['sentiment'] > 0])

negative = len(df[df['sentiment'] < 0])

neutral = len(df[df['sentiment'] == 0])

print('Total Comments:', total)

print('Positive Comments:', positive, f'({positive/total*100:.2f}%)')

print('Negative Comments:', negative, f'({negative/total*100:.2f}%)')

print('Neutral Comments:', neutral, f'({neutral/total*100:.2f}%)')
```

Slip – 26

Q1. Create employee table EMP (eno, ename, designation, salary). Write an AJAX program to

select employee name and print selected employee's details.

Ans:

```
<!-- HTML -->

<select id="employee-list">

<option value="">Select an employee</option>

</select>

<div id="employee-details"></div>

// AJAX (jQuery)

$(document).ready(function() {

$('#employee-list').change(function() {
```

```
var selectedEmployee = $(this).val();

$.ajax({

url: 'empdetails.php',

type: 'POST',

data: { employeeName: selectedEmployee },

dataType: 'json',

success: function(response) {

var detailsHtml = 'Employee Name: ' + response.ename + '<br>' +

'Designation: ' + response.designation + '<br>' +

'Salary: ' + response.salary;

$('#employee-details').html(detailsHtml);

},

error: function(xhr, status, error) { console.log('Error:', error); }

});

});
```

```
<?php // empdetails.php

$conn = pg_connect("host=localhost dbname=db user=user password=pass");

if (!$conn) { die('Connection failed: ' . pg_last_error()); }

$employeeName = $_POST['employeeName'];

$sql = "SELECT * FROM EMP WHERE ename='$employeeName'";

$result = pg_query($conn, $sql);

if (pg_num_rows($result) > 0) {

$employee = pg_fetch_assoc($result);

$response = [

'ename' => $employee['ename'],

'designation' => $employee['designation'],

'salary' => $employee['salary']

];

} else {

echo json_encode($response);

echo "Employee not found";

}

pg_close($conn);

?>
```

Q2. Consider text paragraph: "Hello all, Welcome to Python Programming Academy..."

Preprocess to remove special characters and digits. Generate summary using extractive

summarization.

Ans:

```
import re

from nltk.tokenize import sent_tokenize

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

text = ("Hello all, Welcome to Python Programming Academy. "

"Python Programming Academy is a nice platform to learn new programming skills. "

"It is difficult to get enrolled in this Academy.")

preprocessed_text = re.sub(r'[^a-zA-Z\s]', '', text)

sentences = sent_tokenize(preprocessed_text)

vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(sentences)

sim_matrix = cosine_similarity(tfidf_matrix)

N = 2

top_sentences = sorted(range(len(sim_matrix[-1])),

key=lambda i: sim_matrix[-1][i])[-N:]

summary = ' '.join([sentences[i] for i in top_sentences])
```

```
print(summary)
```

Slip – 27

Q1. Create a web application with Voter details and check proper validation for (name, Age,

nationality): name should be in uppercase only, Age should not be less than 18, Nationality

should be Indian. (Use HTML-AJAX-PHP).

Ans:

```
<!-- HTML file -->

<!DOCTYPE html>

<html>

<head>

<title>Voter Details</title>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

</head>

<body>

<h2>Voter Details</h2>

<form id="voterForm">

<label>Name:</label> <input type="text" id="name" required><br><br>

<label>Age:</label> <input type="number" id="age" required><br><br>

<label>Nationality:</label> <input type="text" id="nationality" required><br><br>

<input type="submit" value="Submit">

</form>

<div id="response"></div>

<script>

$(document).ready(function() {

$('#voterForm').submit(function(event) {

event.preventDefault();

var name = $('#name').val().toUpperCase();

var age = $('#age').val();

var nationality = $('#nationality').val();

$.ajax({

url: 'voter.php',

method: 'POST',

data: { name: name, age: age, nationality: nationality },

success: function(response) { $('#response').html(response); }

});

});

});

</script>

</body>

</html>

<?php // voter.php

$name = $_POST['name'];

$age = $_POST['age'];

$nationality = $_POST['nationality'];

if (preg_match('/[^A-Z]/', $name)) {

echo 'Name should be in uppercase letters only.';

} elseif ($age < 18) {

echo 'Age should not be less than 18 years.';

} elseif (strcasecmp($nationality, 'Indian') != 0) {

echo 'Nationality should be Indian.';

} else {

echo 'Validation successful. Voter details:<br>'

. 'Name: ' . $name . '<br>'

. 'Age: ' . $age . '<br>'

. 'Nationality: ' . $nationality;

}

?>
```

Q2. Create your own transactions dataset and apply the Apriori process on your dataset.

Ans:

```
import random, csv

import pandas as pd

transactions = []
```

```
for i in range(1, 101):

transactions.append([

i,

f"2022-05-{random.randint(1,28):02d}",

random.randint(1, 10),

random.choice(["A", "B", "C"]),

round(random.uniform(10.0, 100.0), 2),

random.randint(1, 10)

])

with open('transactions.csv', 'w', newline='') as csvfile:

writer = csv.writer(csvfile)

writer.writerow(["Transaction ID","Transaction Date","Customer ID","Item ID","Item Price","Quantity"])

for t in transactions:

writer.writerow(t)

df = pd.read_csv('transactions.csv')

df['Item Price'] = pd.to_numeric(df['Item Price'])

df['Sales'] = df['Item Price'] * df['Quantity']

total_sales = df.groupby('Customer ID')['Sales'].sum().reset_index()

print(total_sales)
```

Slip – 28

Q1. Write a PHP script using AJAX concept to check whether username and password are valid

or invalid. (Use Database to store username and password).

Ans:

```
<!-- HTML file -->

<!DOCTYPE html>

<html>

<head>

<title>Login</title>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

<script>

$(document).ready(function() {

$("#login-form").submit(function(event) {

event.preventDefault();

var username = $("#username").val();

var password = $("#password").val();

$.ajax({

url: 'check_login.php',

type: 'post',

data: { username: username, password: password },

success: function(response) {

if (response == "valid") {

window.location.href = "dashboard.php";

} else {

alert("Invalid username or password");

}

}

});

});

});

</script>

</head>

<body>

<h2>Login</h2>

<form id="login-form" method="post">

<label>Username:</label>

<input type="text" name="username" id="username"><br><br>

<label>Password:</label>

<input type="password" name="password" id="password"><br><br>

<input type="submit" value="Login">

</form>

</body>
```

```
</html>

<?php // check_login.php

$conn = mysqli_connect('localhost', 'username', 'password', 'database_name');

if (!$conn) { die('Connection failed: ' . mysqli_connect_error()); }

$username = $_POST['username'];

$password = $_POST['password'];

$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";

$result = mysqli_query($conn, $sql);

echo (mysqli_num_rows($result) > 0) ? "valid" : "invalid";

mysqli_close($conn);

?>
```

Q2. Build a simple linear regression model for Car Dataset.

Ans:

```
from sklearn.linear_model import LinearRegression

mileage = [[10],[20],[30],[40],[50],[60],[70],[80]]

price = [24, 19, 17, 13, 10, 7, 5, 2]

reg = LinearRegression().fit(mileage, price)

print('Intercept:', reg.intercept_)

print('Coefficient:', reg.coef_[0])

new_mileage = [[25], [45], [65]]

predicted_price = reg.predict(new_mileage)

print('Predicted prices:', predicted_price)
```

Slip – 29

Q1. Write a PHP script to design a form to accept a number from the user. Perform operations:

1) Fibonacci Series. 2) Sum of digits of that number. (Use self-processing page concept.)

Ans:

```
<!DOCTYPE html>

<html>

<head><title>Number Operations</title></head>

<body>

<h1>Number Operations</h1>

<?php

$num = $op = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {

$num = test_input($_POST["num"]);

$op = test_input($_POST["op"]);

switch ($op) {

case "fib":

$result = fibonacci($num);

echo "<p>Fibonacci series of $num numbers: $result</p>";

break;

case "sum":

$result = sumOfDigits($num);

echo "<p>Sum of digits in $num is: $result</p>";

break;

default:

echo "<p>Invalid operation selected</p>";

}

function test_input($data) {

return htmlspecialchars(stripslashes(trim($data)));

}

}

function fibonacci($num) {

$first = 0; $second = 1; $result = "";

for ($i = 0; $i < $num; $i++) {

$result .= $first . " ";

$third = $first + $second;

$first = $second;

$second = $third;

}

return $result;
```

```
}

function sumOfDigits($num) {

$sum = 0;

while ($num > 0) {

$sum += $num % 10;

$num = (int)($num / 10);

}

return $sum;

}

?>

<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>">

<label>Enter a number:</label>

<input type="number" name="num" required><br><br>

<label>Select an operation:</label>

<select name="op" required>

<option value="">-- Select --</option>

<option value="fib">Fibonacci Series</option>

<option value="sum">Sum of Digits</option>

</select><br><br>

<input type="submit" value="Submit">

</form>

</body>

</html>
```

Q2. Build a logistic regression model for Student Score Dataset.

Ans:

```
import pandas as pd

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

data = pd.read_csv('student_scores.csv')

X = data.iloc[:, :-1].values

y = data.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

classifier = LogisticRegression()

classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Slip – 30

Q1. Create a XML file which gives details of books in "Bookstore" from categories: 1) Yoga 2)

Story 3) Technical. Each book has Book_Title, Book_Author, Book_Price. Save as

"Bookcategory.xml".

Ans:

```
<?xml version="1.0" encoding="UTF-8"?>

<!-- Bookcategory.xml -->

<Bookstore>

<Yoga>

<Book>

<Book_Title>Light on Yoga</Book_Title>

<Book_Author>B.K.S. Iyengar</Book_Author>

<Book_Price>20.99</Book_Price>

</Book>

<Book>

<Book_Title>The Yoga Bible</Book_Title>

<Book_Author>Christina Brown</Book_Author>

<Book_Price>15.50</Book_Price>

</Book>

</Yoga>

<Story>

<Book>

<Book_Title>The Alchemist</Book_Title>

<Book_Author>Paulo Coelho</Book_Author>
```

<Book_Price>12.99</Book_Price>

</Book>

<Book>

<Book_Title>The Da Vinci Code</Book_Title>

<Book_Author>Dan Brown</Book_Author>

<Book_Price>14.75</Book_Price>

</Book>

</Story>

<Technical>

<Book>

<Book_Title>Python for Data Science Handbook</Book_Title>

<Book_Author>Jake VanderPlas</Book_Author>

<Book_Price>28.99</Book_Price>

</Book>

<Book>

<Book_Title>Cracking the Coding Interview</Book_Title>

<Book_Author>Gayle Laakmann McDowell</Book_Author>

<Book_Price>23.50</Book_Price>

</Book>

</Technical>

</Bookstore>

Q2. Create the transactions dataset:

[['eggs','milk','bread'],['eggs','apple'],['milk','bread'],['apple','milk'],['milk','apple','bread']].

Convert categorical values to numeric. Apply Apriori algorithm to generate frequent itemsets

and association rules.

Ans:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

transactions = [

['eggs', 'milk', 'bread'],

['eggs', 'apple'],

['milk', 'bread'],

['apple', 'milk'],

['milk', 'apple', 'bread']

]

# Map items to unique numeric values

item_to_num = {'eggs': 1, 'milk': 2, 'bread': 3, 'apple': 4}

numeric_transactions = [

[item_to_num[item] for item in transaction]

for transaction in transactions

]

print("Numeric transactions:", numeric_transactions)

# Apply Apriori

te = TransactionEncoder()

te_ary = te.fit_transform(transactions)

df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)

rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

print("\nFrequent Itemsets:\n", frequent_itemsets)

print("\nAssociation Rules:\n", rules)
```